

Code Assessment of the Onyx CCIP Wallet Smart Contracts

PUBLIC

Date [May 08, 2026](#)

Produced for



By



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	5
4	Trust Model	8
5	System Considerations	9
6	Terminology	10
7	Findings	11
8	Limitations and use of report	14

1 Executive Summary

Dear Enzyme Team,

Thank you for trusting us to help Enzyme Foundation with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Onyx CCIP Wallet according to [Scope](#) to support you in forming an opinion on their security risks.

Enzyme Foundation implements a cross-chain wallet integration for the Onyx smart contract protocol. The system uses CCIP to transfer messages and tokens between blockchains.

The most critical subjects covered in our audit are functional correctness, fund isolation and fault tolerance. Security regarding all the aforementioned subjects is high.

In summary, we find that the codebase provides a high level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Onyx CCIP Wallet repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	24 April 2026	10c416bc17720cadbc7f4ae5b736e5d8c1d14768	Initial review
2	5 May 2026	36ce6829a72aefd5b2eb72cc40100bd50a05ae9c	After Intermediate Report

For the Solidity smart contracts, the compiler version `0.8.28` was chosen.

```
src/
├── ccip/
│   └── DepositorWallet.sol
├── factories/
│   └── DeterministicBeaconFactory.sol
└── components/
    └── ccip/
        └── WalletsManager.sol
```

2.1.1 Excluded from scope

Third-party libraries (such as OpenZeppelin and Chainlink CCIP) and any files not explicitly listed above are out of scope. Chainlink CCIP itself, including the router, on-ramp, off-ramp, and token pools, is out of scope and assumed to function according to its specifications.

3 System Overview

This system overview describes the initially received version (`Version 1`) of the contracts as defined in the [Assessment Overview](#).

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Enzyme Foundation offers an extension to Enzyme Onyx, a set of smart contracts to tokenize on- and off-chain value. The extension enables cross-chain token management through Chainlink CCIP. It revolves around per-user wallets that are deterministically deployed on a destination chain whenever a user sends tokens cross-chain. The `WalletsManager` acts as the central orchestrator, receiving inbound CCIP messages, deploying or locating user wallets, forwarding tokens, and executing arbitrary calls on behalf of users. The `DeterministicBeaconFactory` provides the deployment mechanism, using `CREATE2` to ensure wallet addresses are predictable before deployment. Each `DepositorWallet` is a beacon proxy instance that holds tokens for a specific user and can send them back to the user's source chain via CCIP.

The typical flow starts when a user sends a CCIP message from a source chain. The `WalletsManager` receives this message, deploys or retrieves the user's `DepositorWallet`, transfers the received tokens into it, and optionally executes encoded calls (e.g. unwrapping of native wrapped tokens, deposits into the Onyx protocol, or other DeFi interactions). If the message includes a token return request, the wallet builds a CCIP message containing its full token balances and sends them back to the user on the source chain.

3.1 DeterministicBeaconFactory

The `DeterministicBeaconFactory` is a generic factory that deploys BeaconProxy instances at deterministic addresses using `CREATE2`. It serves as the beacon that all deployed proxies point to for their implementation address. The owner can update the implementation at any time via `setImplementation()`, upgrading all proxies in a single transaction. The factory tracks all deployed instances in an `isInstance()` mapping.

The salt used for `CREATE2` is derived from `keccak256(abi.encode(msg.sender, _salt))`, which means different deployers (e.g. different `WalletsManager` instances) will produce different addresses even with the same salt. The factory also exposes `computeProxyAddress()` to predict the address of a proxy before deployment, which the `WalletsManager` uses to check whether a wallet already exists.

3.2 WalletsManager

The `WalletsManager` is a `CCIPReceiver` that serves as the entry point for cross-chain messages and the management layer for all `DepositorWallet` instances. When an inbound CCIP message arrives, it extracts the sender identity and source chain selector, then calls the factory to either compute the existing wallet address or deploy a new one. The `CREATE2` salt passed to the factory is `keccak256(abi.encode(sourceChainSelector, user))`, so every distinct sender on a given source chain maps to exactly one wallet address on the destination chain. Tokens from the message are transferred to the wallet, and the optional message data is processed in a try-catch block so that failures in call execution do not revert the token transfer. This allows tokens to safely arrive at the wallet even if the hook data sent by the user is malformed or one of the executed calls fails. In this case, the user can always choose to send another message with corrected hook data.

Message data is ABI-decoded into four components.

Component	Purpose
<code>calls</code>	Arbitrary calls to execute from the wallet (e.g. token swaps)

Component	Purpose
<code>tokensToReturn</code>	Token addresses to send back to the user via CCIP
<code>extraArgs</code>	CCIP extra args for the return message
<code>feeTokenForReturn</code>	Token used to pay CCIP fees for the return trip

The contract also provides `batchSendTokensViaCCIP()` , an admin function that triggers token returns from multiple wallets in a single transaction, paying CCIP fees with native currency. Any leftover native balance is refunded to the caller.

3.3 DepositorWallet

The `DepositorWallet` is an initializable beacon proxy implementation that acts as an isolated per-user wallet. Each instance is bound to a specific user (identified by their encoded source chain address) and a CCIP chain selector. Only its parent `WalletsManager` can invoke actions on it, enforced by the `onlyWalletsManager` modifier.

The wallet has three core capabilities.

- **executeCalls()** runs a batch of arbitrary calls (target, data, value) from the wallet's context, enabling interactions with external protocols
- **buildTokenReturnMessage()** constructs a CCIP message containing the wallet's full balance of specified tokens and computes the associated fee
- **sendCCIPMessage()** approves tokens to the CCIP router and dispatches the message to the user's source chain

The wallet uses ERC-7201 namespaced storage to avoid storage collisions in the proxy pattern. It holds an immutable reference to the CCIP router and can receive native currency via its `receive()` function.

3.4 Typical Flow -- Cross-Chain Deposit and Share Return

A concrete example of the system in action is a user depositing tokens from a source chain into the Onyx vault on the destination chain and receiving minted shares back.

1. The user constructs a CCIP message on the source chain. The message carries the deposit tokens (e.g., USDC) and encodes the intended actions in its data field. The encoded data includes a `calls` array that will approve the deposit asset to the `SyncDepositHandler` and then call `deposit` on it, a `tokensToReturn` array containing the Shares token address, CCIP `extraArgs` , and a `feeTokenForReturn` .
2. The CCIP Router on the destination chain delivers the message to the `WalletsManager` . The `WalletsManager` extracts the user's identity and source chain selector, then computes the deterministic wallet address. If this is the user's first interaction, the `DeterministicBeaconFactory` deploys a new `BeaconProxy` as the user's `DepositorWallet` . Otherwise, the existing wallet is reused.
3. The `WalletsManager` transfers the bridged tokens (e.g. USDC) from itself to the user's `DepositorWallet` .
4. The wallet executes the encoded calls in sequence. First it approves the deposit asset to the `SyncDepositHandler` . Then it calls `deposit()` , which pulls the approved tokens from the wallet into the Shares contract and mints shares back to the wallet.
5. Still within `processMessageData()` , the `WalletsManager` sees that `tokensToReturn` is non-empty. It calls `buildTokenReturnMessage()` on the wallet, which reads the wallet's full balance of each specified

token (in this case the newly minted shares) and constructs a CCIP return message addressed to the user on the source chain. The CCIP fee is computed and deducted from the fee token's amount in the message.

6. The `WalletsManager` calls `sendCCIPMessage()` on the wallet. The wallet approves the share tokens to the CCIP Router and dispatches the message. The shares are bridged back to the user on the source chain.

If the call execution fails (e.g. due to slippage or a stale share price), the try-catch in the `WalletsManager` ensures the bridged tokens still land safely in the user's `DepositorWallet`. The tokens can then be returned to the user via a subsequent `batchSendTokensViaCCIP()` call by an admin, or through a new CCIP message from the user.

4 Trust Model

4.1 Owner (Global.owner)

Used by `DeterministicBeaconFactory`. Ownership is resolved dynamically through the `Global` contract. The owner can call `setImplementation()` on the factory, changing the beacon implementation for all deployed `DepositorWallet` proxies at once.

- **Trust level** -- Fully trusted
- **Worst case** -- Can set a malicious implementation on the factory, taking over all existing `DepositorWallet` proxies and stealing any tokens held in them.

4.2 Shares Owner and Admins

Used by `WalletsManager`. The check delegates to the Shares contract associated with the `WalletsManager`'s proxy. These roles can call `batchSendTokensViaCCIP()` to trigger token returns from multiple wallets, paying CCIP fees with native currency.

- **Trust level** -- Partially trusted
- **Worst case** -- Can force-send all tokens from any `DepositorWallet` back to its registered user via CCIP. Cannot redirect tokens to arbitrary addresses since the destination is fixed per wallet.

4.3 CCIP Router

The Chainlink CCIP Router is the only address that can invoke `ccipReceive()` on the `WalletsManager`. Inbound CCIP messages trigger wallet deployment, token transfers, call execution, and optional token returns.

- **Trust level** -- Fully trusted
- **Worst case** -- A compromised router could forge messages with arbitrary sender identities and token amounts, causing wallets to be deployed or calls to be executed under false pretenses.

4.4 End Users

End users interact with the system by sending CCIP messages from a source chain. They have no direct write access to any of the three contracts on the destination chain. Their source chain address is encoded in the CCIP message and used as the wallet identifier. Users can call view functions such as `computeWalletAddress()` and the various state getters on `DepositorWallet`.

- **Trust level** -- Untrusted

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 Fee-on-Transfer Tokens Are Not Supported

System Consideration	Version 1
----------------------	-----------

The contracts assume an ERC20 transfer of `amount` increases the recipient's balance by exactly `amount`. Fee-on-transfer (and rebasing) tokens violate this assumption.

`WalletsManager._ccipReceive()` forwards inbound CCIP tokens with `safeTransfer(wallet, tokenAmounts[i].amount)`, using the amount declared by the CCIP message rather than the balance actually received.

`DepositorWallet.sendCCIPMessage()` approves the router for `_message.tokenAmounts[i].amount`, which `buildTokenReturnMessage()` reads from `balanceOf(this)`. With a fee-on-transfer token, the CCIP pool receives less than what the outgoing message announces.

Fee-on-transfer and rebasing tokens are therefore not supported.

SC2 No Timelock on DepositorWallet Upgrades

System Consideration	Version 1
----------------------	-----------

`DeterministicBeaconFactory.setImplementation()` takes effect immediately and applies to every deployed `DepositorWallet`. The owner can therefore swap the implementation without any window for users to exit. No on-chain mitigation is in place against this.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

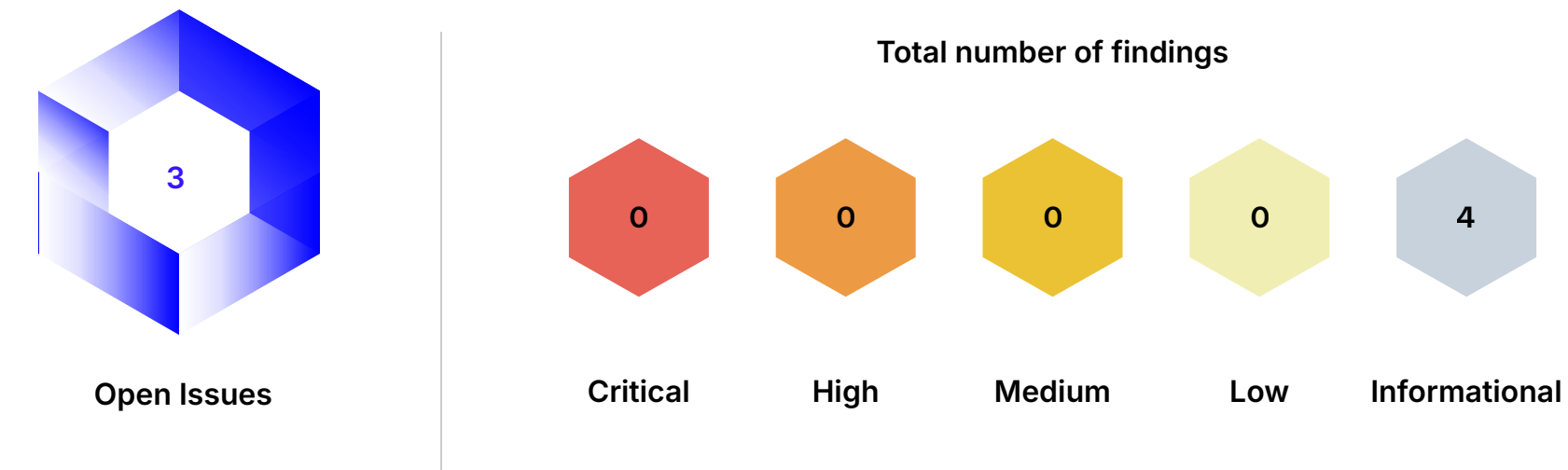
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



7.2 Descriptions

#001 DepositorWallet Implementation Does Not Disable Initializers

Informational	Version 1	Code Corrected 
---------------	-----------	--

`DepositorWallet` inherits from `Initializable` and is intended to be used only behind a `BeaconProxy` deployed by `DeterministicBeaconFactory`.

The constructor does not call `_disableInitializers()`. Any account can therefore call `init()` directly on the implementation address and become its `walletsManager`.

As `walletsManager` of the implementation, the attacker can call `executeCalls` to drain any ETH or tokens accidentally sent to the implementation address, and call `sendCCIPMessage()` to emit CCIP messages from it.

Code corrected:

The constructor of `DepositorWallet` now calls `_disableInitializers()`.

#002 DeterministicBeaconFactory.deployProxy Is Permissionless

Informational	Version 1	Risk Accepted 
---------------	-----------	---

`deployProxy()` has no access control: anyone can deploy a `BeaconProxy` with arbitrary `_initData` and any salt. The salt is namespaced by `msg.sender` (`keccak256(abi.encode(msg.sender, _salt))`), so deployments cannot collide with the addresses computed by `WalletsManager` (whose deployments use `WalletsManager` as `msg.sender`). However, attacker-deployed proxies are still recorded in `isInstance`, which mixes user wallets with arbitrary third-party proxies. Any consumer of `isInstance` could be misled into trusting attacker-controlled proxies.

Risk accepted:

Enzyme Foundation accepts the risk with the following statement:

It follows the pattern of other factories that we use, so we would like to keep it consistent.

#003 Events Emit No indexed Parameters

Informational	Version 1	Risk Accepted 
---------------	-----------	---

No parameter on `CallExecuted`, `CCIPMessageSent`, `ChainSelectorSet`, `UserSet`, `WalletsManagerSet`, `CCIPMessageProcessed`, or `MessageDataProcessingFailed` is marked as `indexed`. It is recommended to index the relevant event parameters to allow integrators and dApps to quickly search them.

Risk accepted:

Enzyme Foundation accepts the risk with the following statement:

We decided not to use index parameter in the Onyx protocol, as we don't use indexed in practice.

#004 Possible Revert When Fee Tokens Are Sent Back

Informational

Version 1

Risk Accepted ⓘ

When a user sends some tokens to the wallet that should be used as fee token, they might slightly overestimate the amount and mark the token as a return token in order to get back any residual value. However, if the fee turns out to be the exact amount the user has sent, this will result in the return token amount to be set to 0. When `Router.ccipSend()` then tries to transfer 0 tokens to the respective pool, this may result in a revert depending on the token implementation.

Risk accepted:

Enzyme Foundation accepts the risk with the following statement:

We were aware of this edge case during development, and we came to conclusion it is highly unlikely that the fee amount will exactly match the amount sent by the user.

8 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.